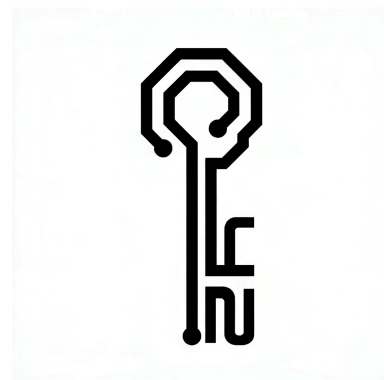


Национальный исследовательский ядерный университет «МИФИ»

Институт интеллектуальных кибернетических систем

Кафедра № 42 «Криптология и кибербезопасность»



ОТЧЕТ

О проведении анализа алгоритма умножения матриц в параллельных сетях

Выполнили: Доценко В. А., Дзюба С.Д.

Группа: Б23-505

Преподаватель: Куприяшин М.А.

Москва — 2025

Оглавление

1. Описание рабочей среды	3
2. Введение	3
3. Последовательный алгоритм	4
3.1. Описание последовательного алгоритма	4
3.2. Блок-схема	5
3.3. Анализ алгоритма	6
4. Параллельный алгоритм	6
5. Результаты экспериментов	8
5.1. Последовательный алгоритм	8
5.2. Параллельный алгоритм	9
5.3. Сравнение параллельного алгоритма с алгоритмами других команд . .	13
6. Заключение	18
7. Приложение	21
7.1. Программные коды	21
7.2. Таблицы	31

1. Описание рабочей среды

Модель процессора: AMD EPYC 7713

Количество ядер / потоков: 32 / 32.

Объем оперативной памяти: 50 ГБ.

Разрядность и версия ОС: Ubuntu 24.04.2 LTS x86_64

Среда разработки: VS Code, clang 18.1.3.

Версия OpenMP: 5.1 (_OPENMP = 202011).

2. Введение

Умножение матриц в параллельных сетях является одной из фундаментальных задач параллельных вычислений, широко применяемой в научных, инженерных и аналитических расчётах. Основная цель параллельного умножения матриц — ускорение вычислений за счёт распределения работы между несколькими вычислительными узлами, соединёнными в сеть.

Параллельные сетевые архитектуры позволяют разделить большие матричные операции между множеством процессов, минимизируя время выполнения и повышая эффективность использования ресурсов. В зависимости от структуры сети, способов взаимодействия процессов, а также метода распределения данных, достигается различная степень ускорения и масштабируемости.

Данная работа посвящена исследованию принципов и особенностей реализации умножения матриц в параллельных сетях, включая методы распределения данных, коммуникационные схемы и способы минимизации затрат на обмен сообщениями.

3. Последовательный алгоритм

3.1. Описание последовательного алгоритма

Последовательный алгоритм умножения прямоугольных матриц является базовым подходом, используемым в большинстве современных параллельных и распределённых реализаций. Его суть заключается в последовательном вычислении каждого элемента результирующей матрицы C как скалярного произведения соответствующей строки матрицы A и столбца матрицы B .

Пусть заданы две прямоугольные матрицы:

- A размером $m \times n$,
- B размером $n \times k$.

Тогда результирующая матрица $C = A \times B$ имеет размер $m \times k$, а каждый её элемент вычисляется по формуле:

$$C_{ij} = \sum_{r=1}^n A_{ir} \cdot B_{rj}, \quad i = 1, 2, \dots, m, \quad j = 1, 2, \dots, k$$

Алгоритм реализуется с помощью трёх вложенных циклов:

- внешний цикл — по строкам матрицы A (i от 1 до m);
- средний цикл — по столбцам матрицы B (j от 1 до k);
- внутренний цикл — по элементам строки A и столбца B (r от 1 до n).

Пошаговое описание алгоритма:

1. Инициализация матрицы C нулями размером $m \times k$.
2. Для каждой строки i матрицы A :
 - а. Для каждого столбца j матрицы B :
 - і. Вычислить элемент C_{ij} и записать в матрицу C

3.2. Блок-схема

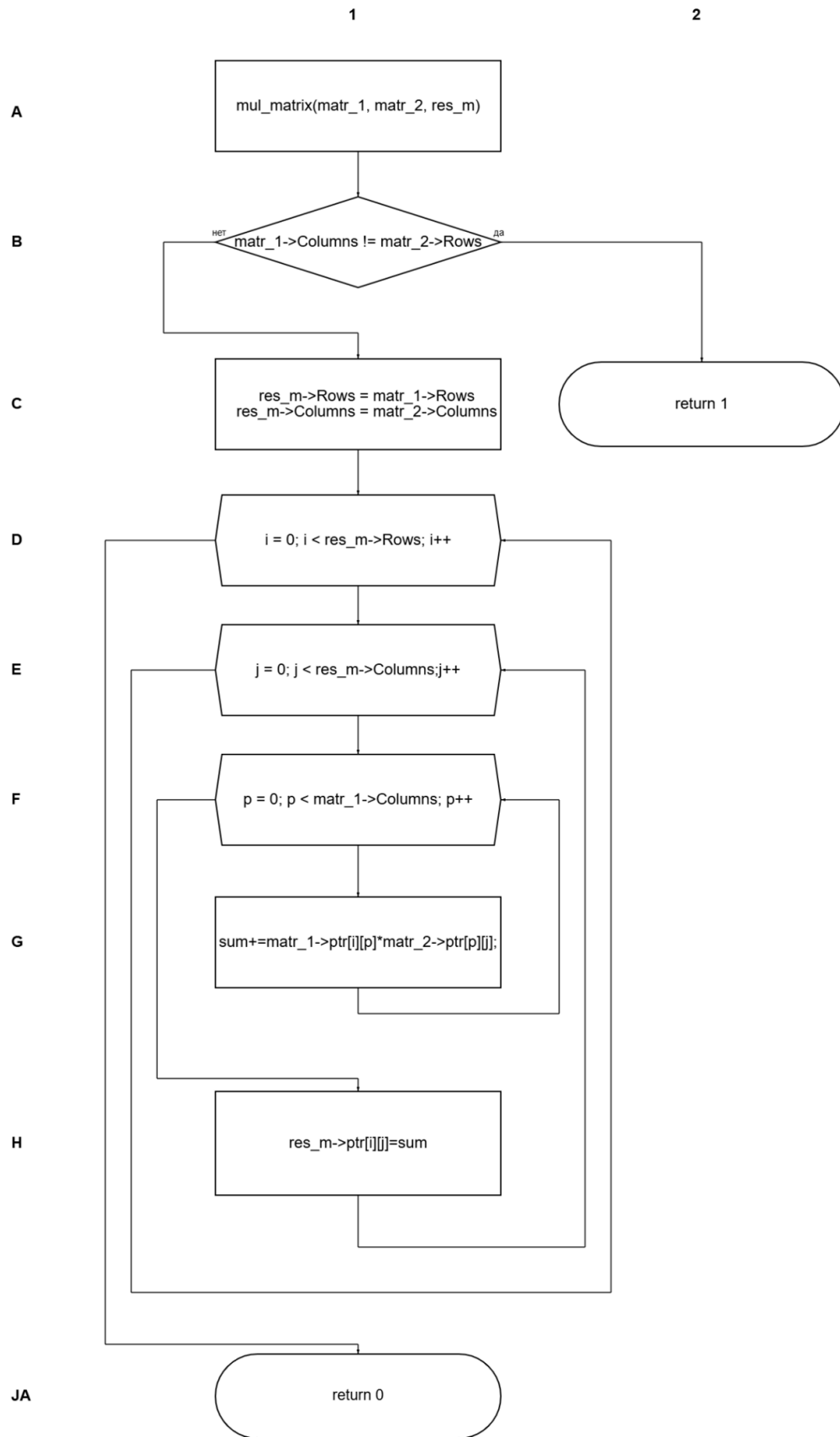


Рисунок 1 – Блок схема последовательного алгоритма

3.3. Анализ алгоритма

Количество выполняемых элементарных операций зависит от размеров матриц A и B :

- Умножения: $m \cdot n \cdot k$
- Сложения: $m \cdot k \cdot (n - 1)$

Следовательно, временная сложность последовательного алгоритма:

$$T(m, n, k) = O(m \cdot n \cdot k)$$

Объём используемой памяти:

$$M(m, n, k) = O(m \cdot n + n \cdot k + m \cdot k)$$

поскольку необходимо хранить три матрицы: A , B и C .

Последовательный алгоритм является вычислительно простым, но при увеличении размеров входных матриц время его работы растёт пропорционально произведению их размеров. Это делает его неэффективным для больших матриц и является основной предпосылкой для перехода к параллельным методам умножения, которые позволяют распределить вычислительную нагрузку между несколькими процессами и существенно сократить время выполнения.

4. Параллельный алгоритм

Параллельное выполнение достигается за счет распараллеливания по строкам матрицы A с использованием директивы OpenMP — `#pragma omp parallel for`. Каждый поток обрабатывает одну или несколько строк матрицы A , независимо вычисляя соответствующие строки результирующей матрицы C . Такой подход обеспечивает:

- отсутствие необходимости в синхронизации между потоками, поскольку каждая строка C формируется уникальным потоком;
- эффективное использование процессорных ядер при больших n ;
- равномерное распределение нагрузки при условии равного числа строк, назначаемых каждому потоку.

Входные данные

- Матрица A размером $m \times n$
- Матрица B размером $n \times k$
- Количество потоков

Выходные данные

- Матрица $C = A \times B$ размером $m \times k$

Доступ к элементам матрицы C не требует атомарных операций, так как каждая строка формируется независимым потоком. Число обращений к общей памяти минимизировано за счет последовательного доступа к элементам строк матрицы A и столбцов матрицы B .

5. Результаты экспериментов

Каждая точка на графиках была получена повторением эксперимента 20 раз.

5.1. Последовательный алгоритм

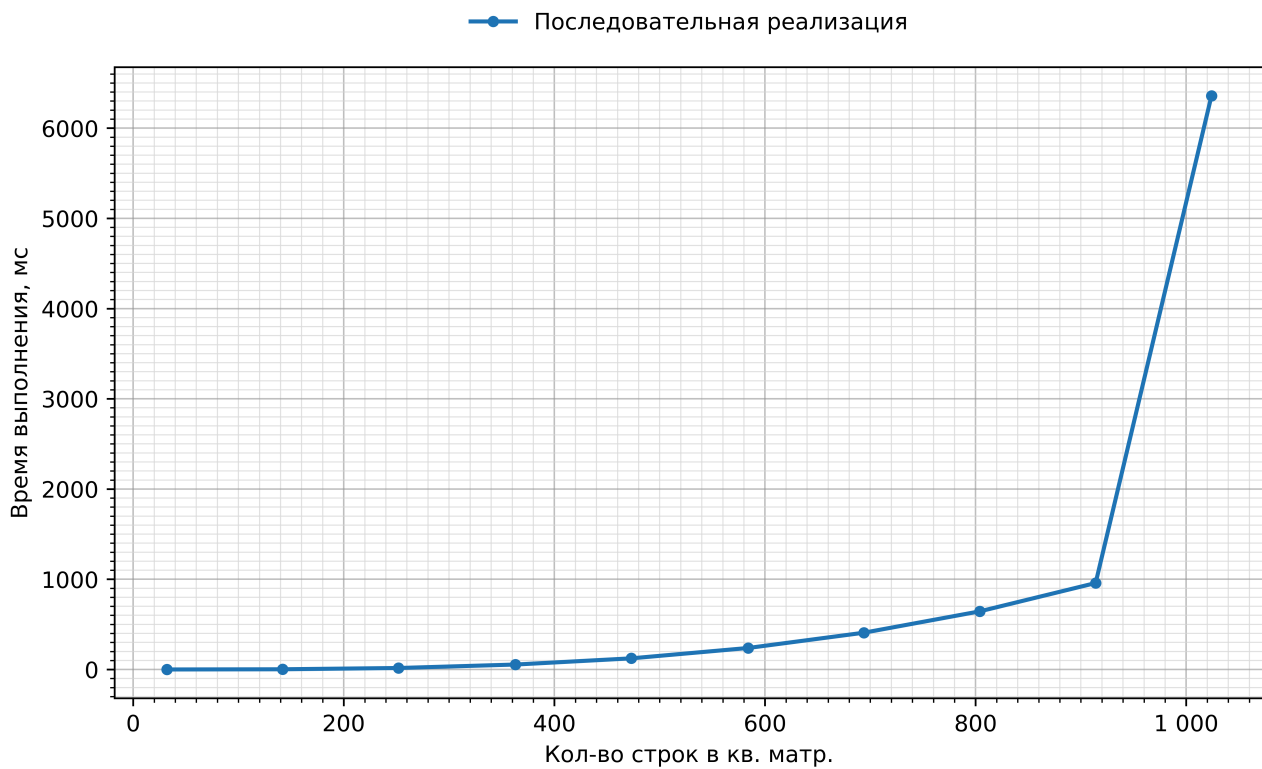


Рисунок 2 – График зависимости времени выполнения перемножения от размерности квадратной матрицы для последовательного алгоритма

5.2. Параллельный алгоритм

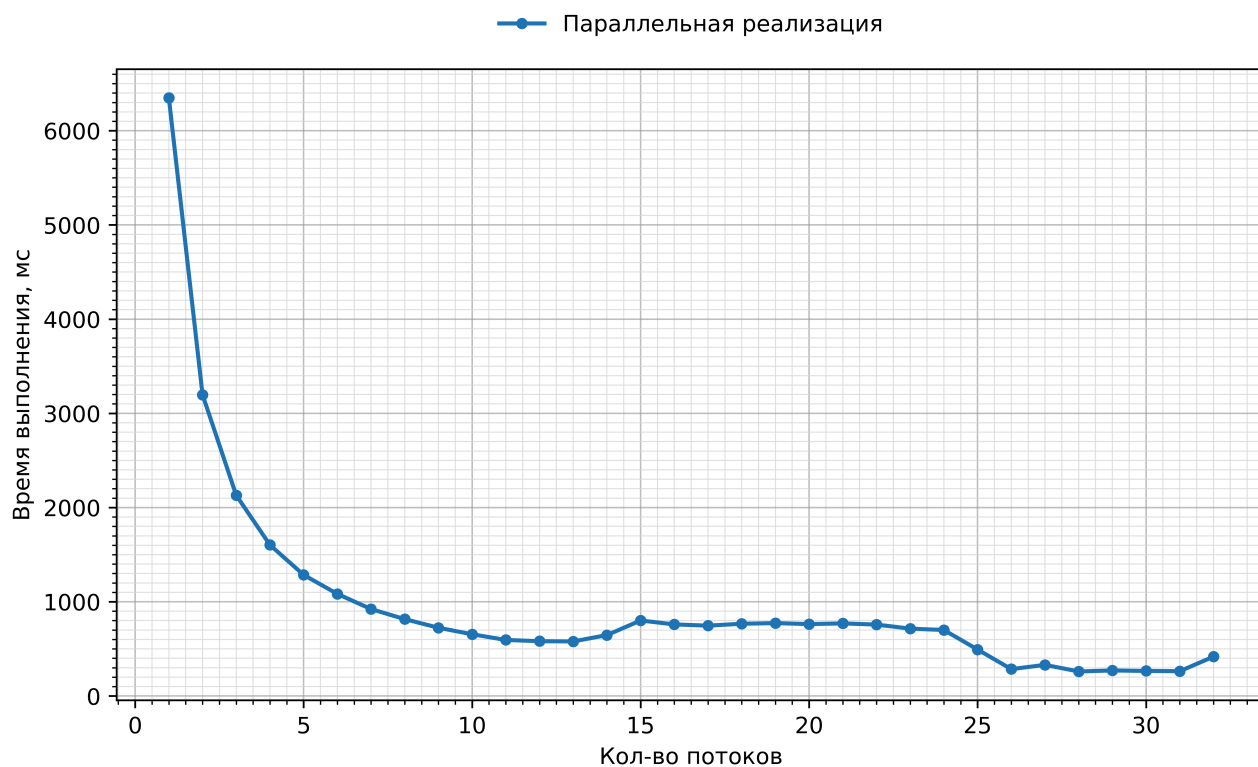


Рисунок 3 – График зависимости времени выполнения перемножения от кол-ва потоков для параллельного алгоритма, размер матрицы 1024x1024



Рисунок 4 – График зависимости ускорения от кол-ва потоков для параллельного алгоритма, размер матрицы 1024x1024

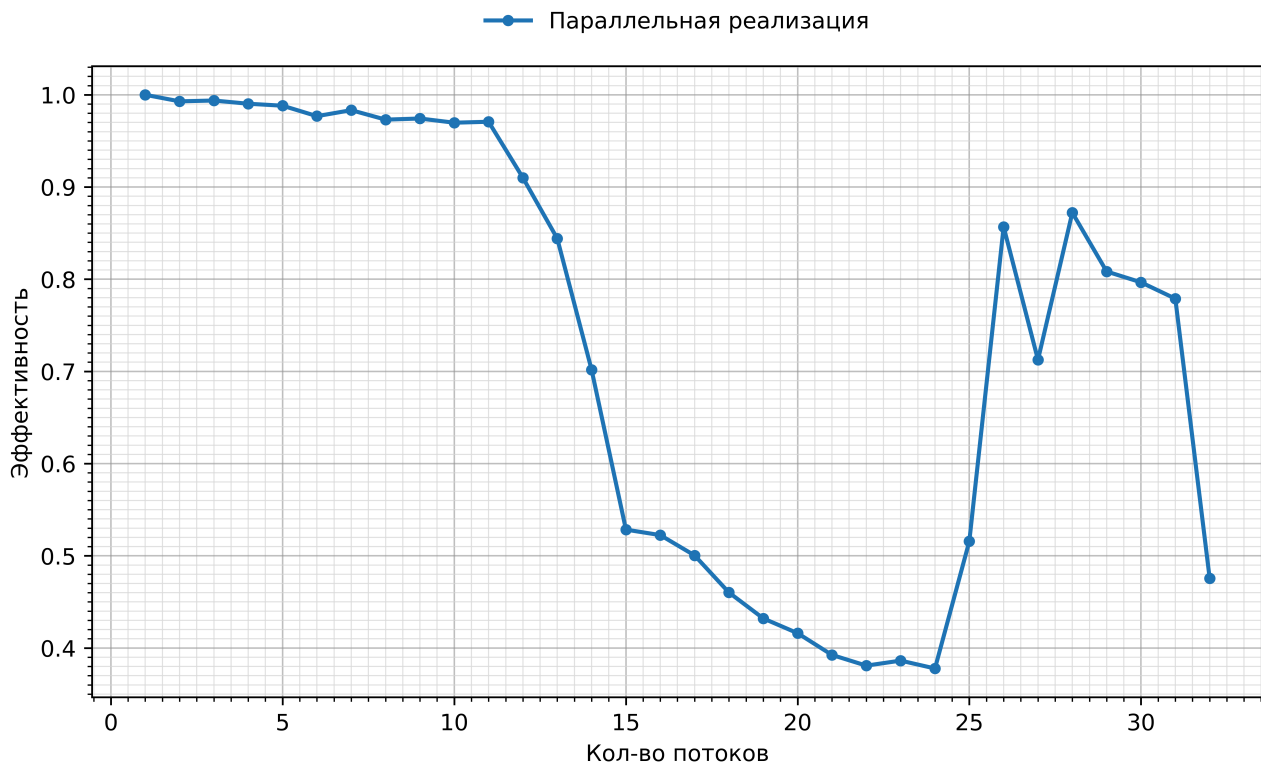


Рисунок 5 – График зависимости эффективности от кол-ва потоков для параллельного алгоритма, размер матрицы 1024x1024

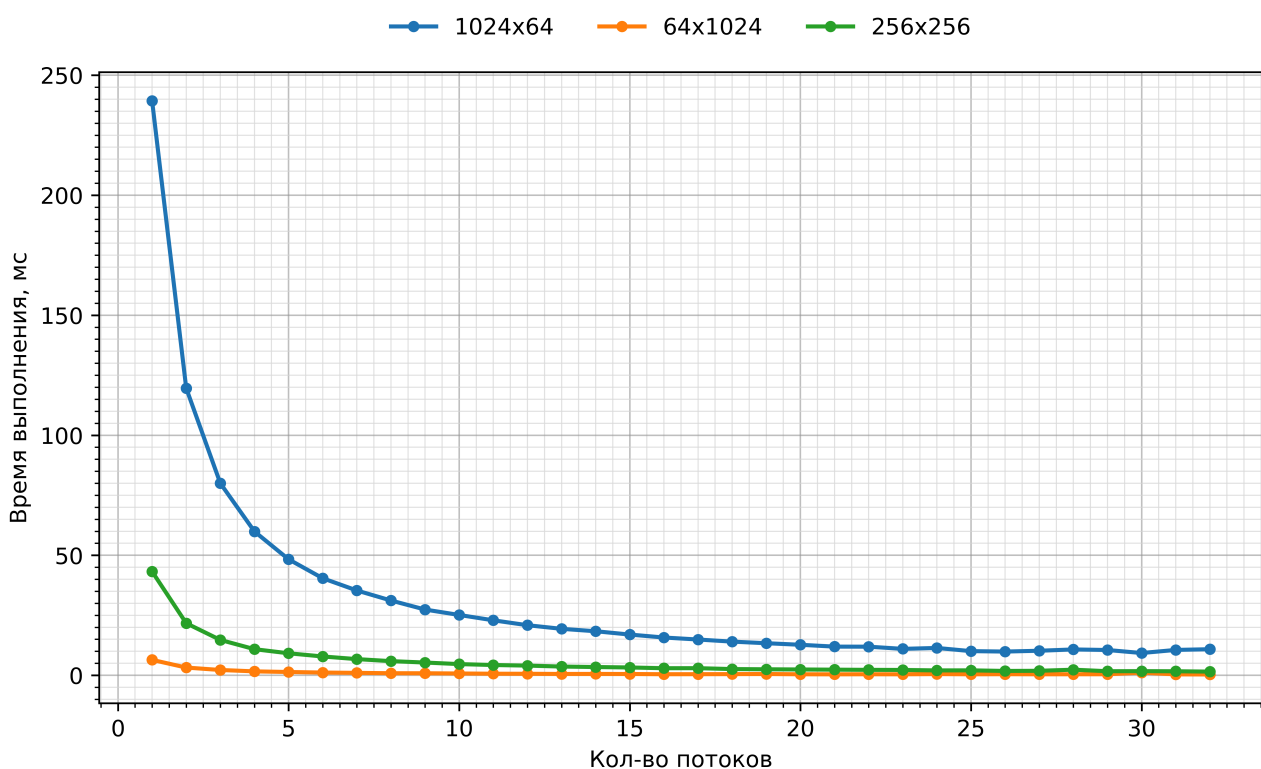


Рисунок 6 – График зависимости времени выполнения от кол-ва потоков для параллельного алгоритма, для разных размеров прямоугольных матриц. Каждая прямоугольная матрица перемножалась на матрицу с размерами, равными транспонированной, что давало различные размеры итоговой матрицы.

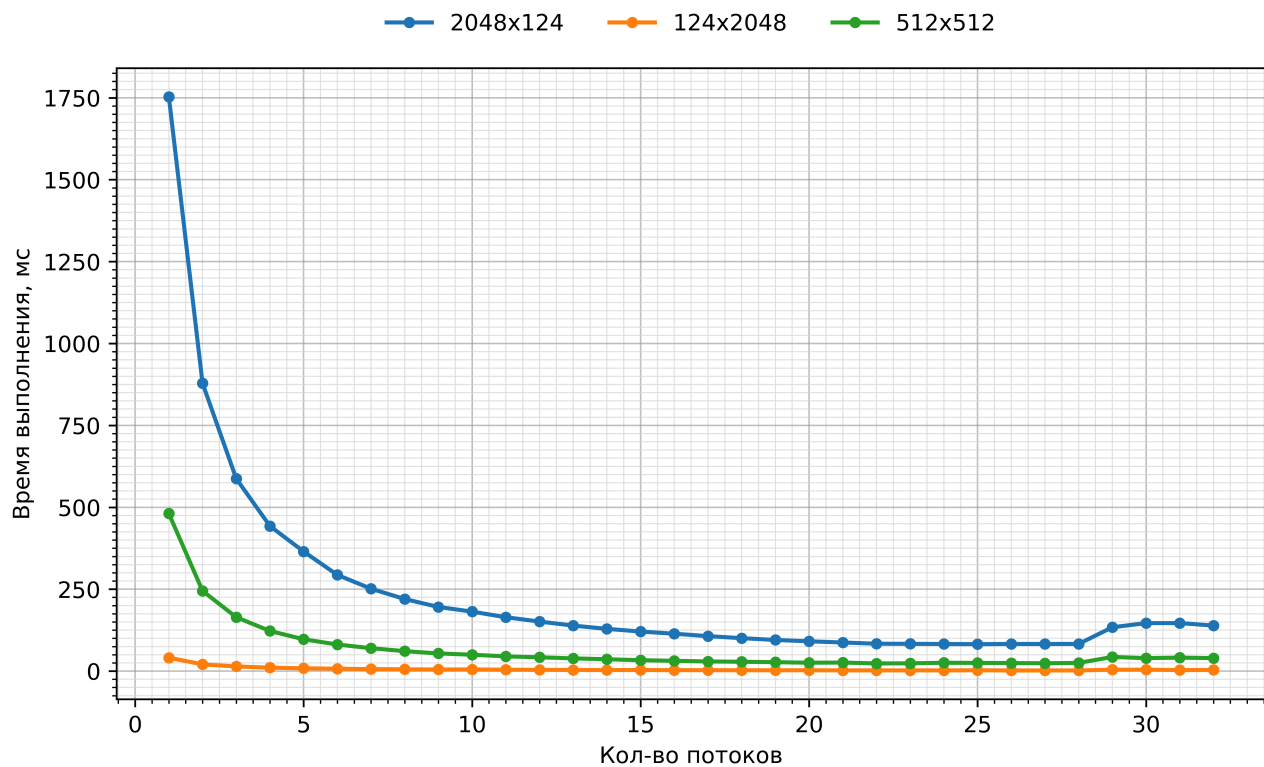


Рисунок 7 – График зависимости времени выполнения от кол-ва потоков для параллельного алгоритма, для разных размеров прямоугольных матриц. Каждая прямоугольная матрица перемножалась на матрицу с размерами, равными транспонированной, что давало различные размеры итоговой матрицы. Увеличенный размер матриц

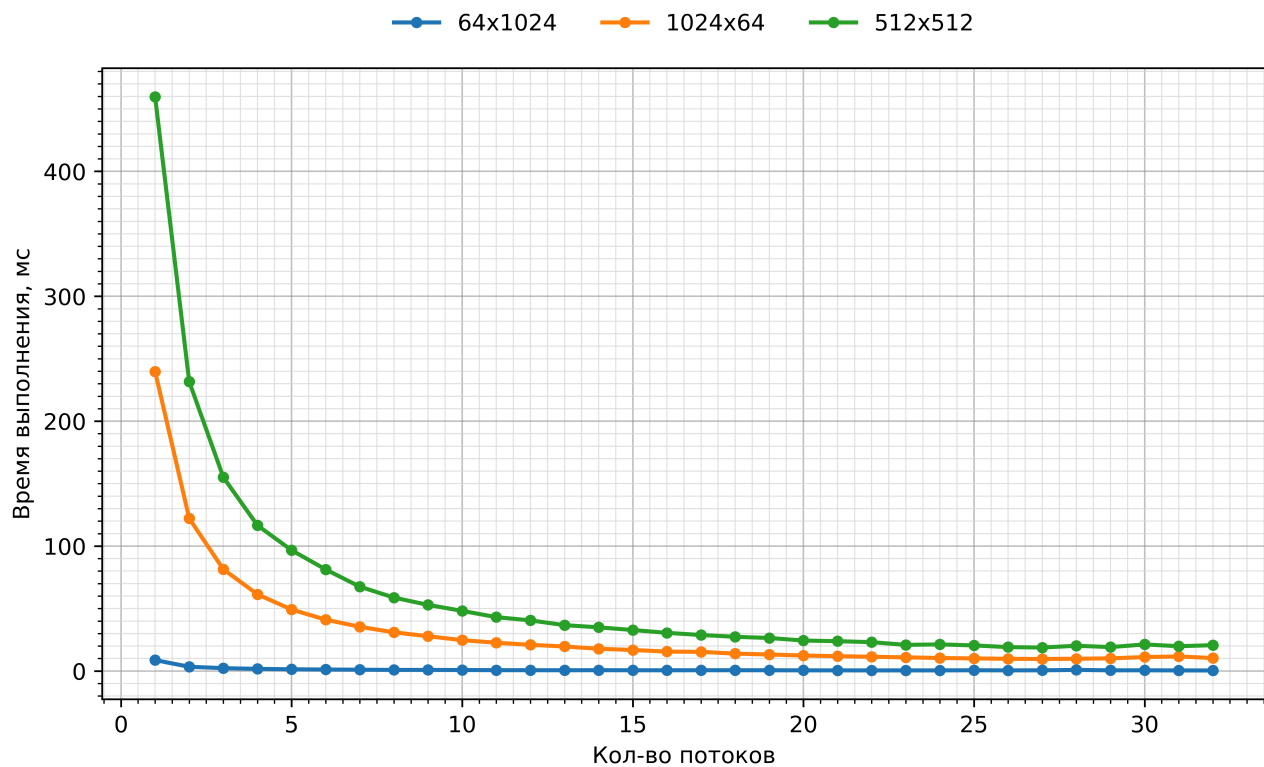


Рисунок 8 – График зависимости времени выполнения от кол-ва потоков для параллельного алгоритма, для разных размеров прямоугольных матриц. Каждая прямоугольная матрица перемножалась на матрицу с размерами, равными транспонированной, что давало различные размеры итоговой матрицы. Матрицы содержали 50% ненулевых элементов

5.3. Сравнение параллельного алгоритма с алгоритмами других команд

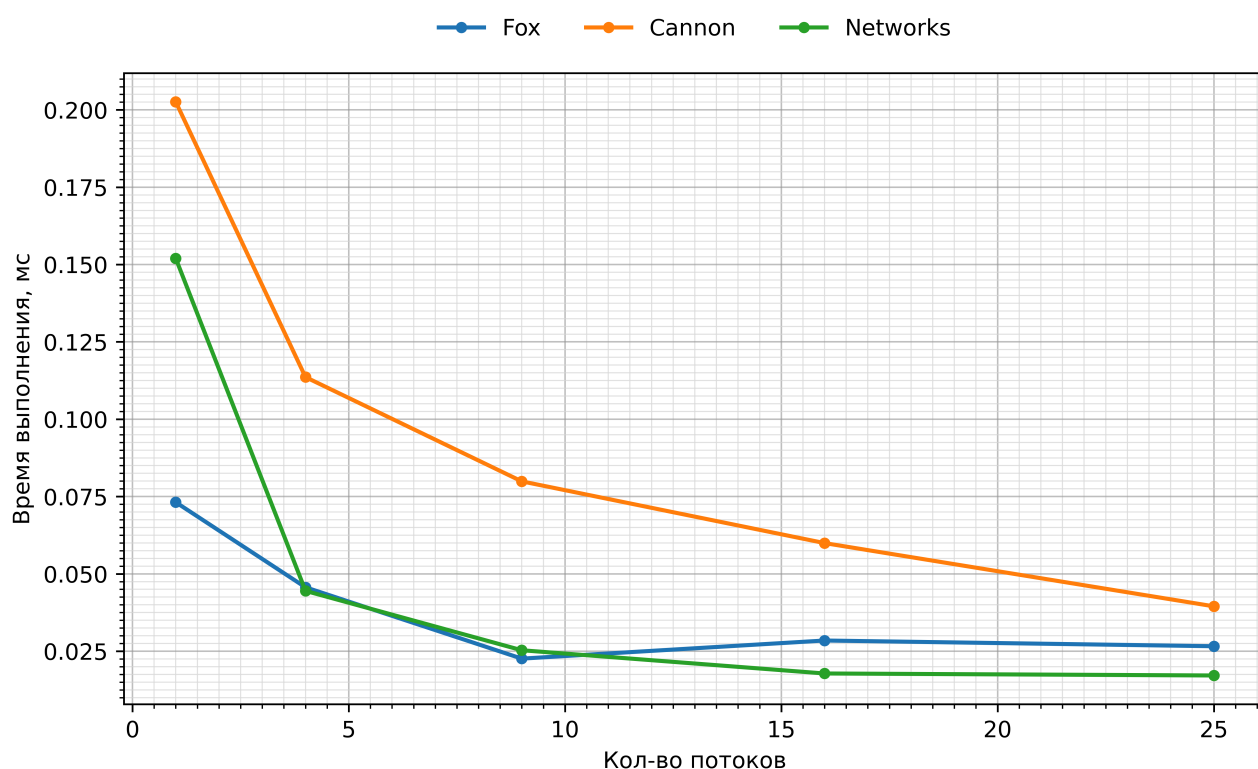


Рисунок 9 – График зависимости времени выполнения перемножения от кол-ва потоков для разных алгоритмов, матрица 60x60

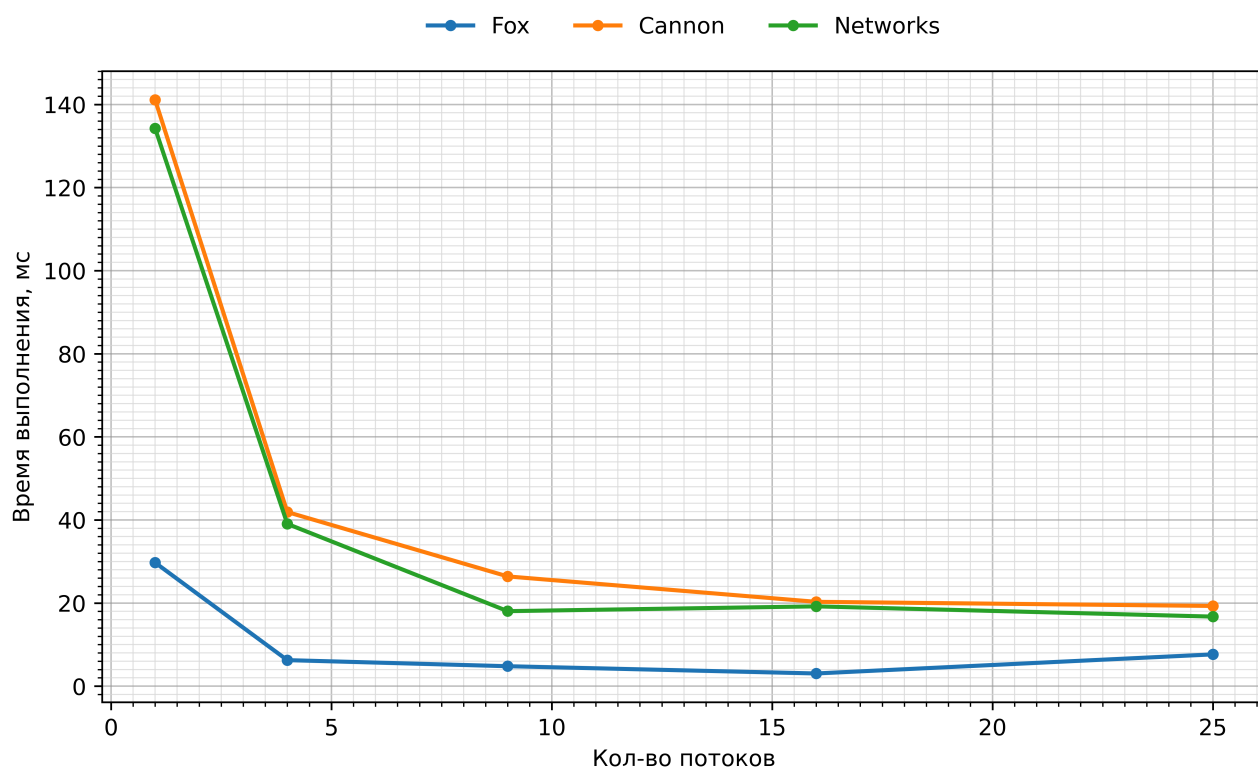


Рисунок 10 – График зависимости времени выполнения перемножения от кол-ва потоков для разных алгоритмов, матрица 480x480

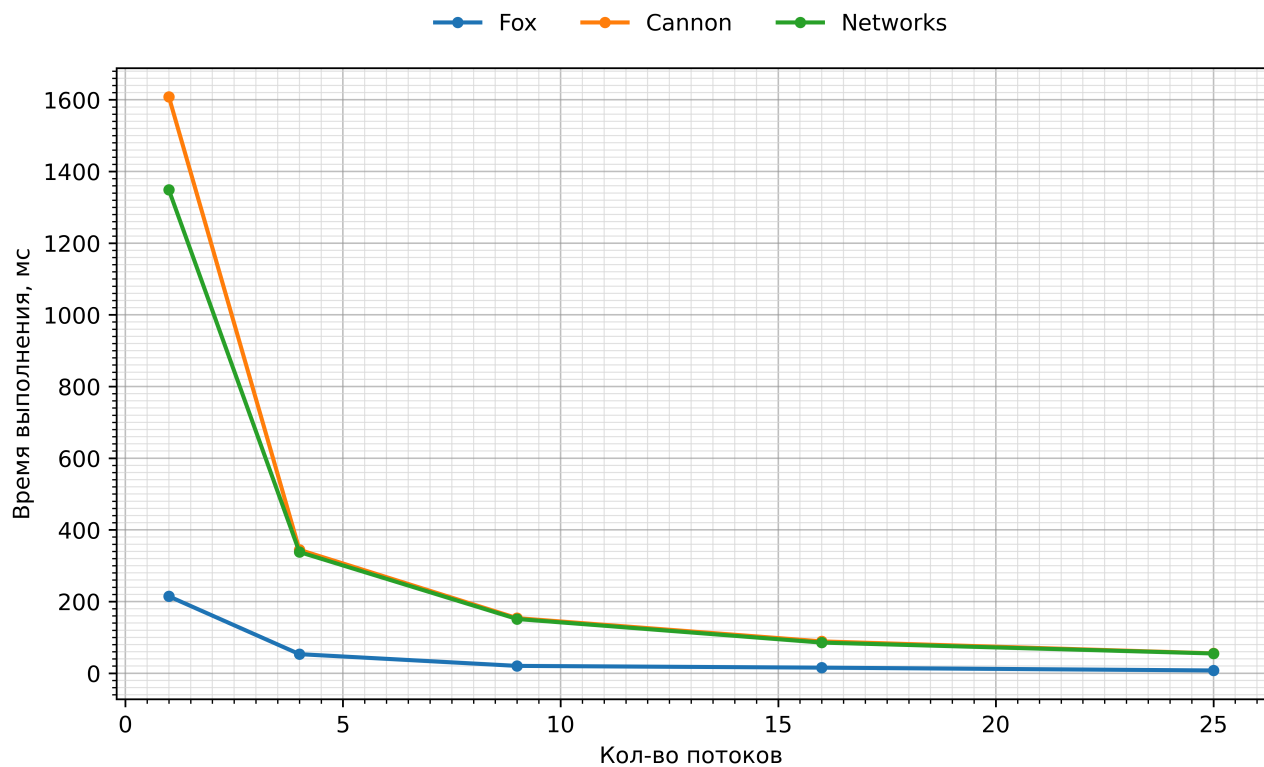


Рисунок 11 – График зависимости времени выполнения перемножения от кол-ва потоков для разных алгоритмов, матрица 1020x1020

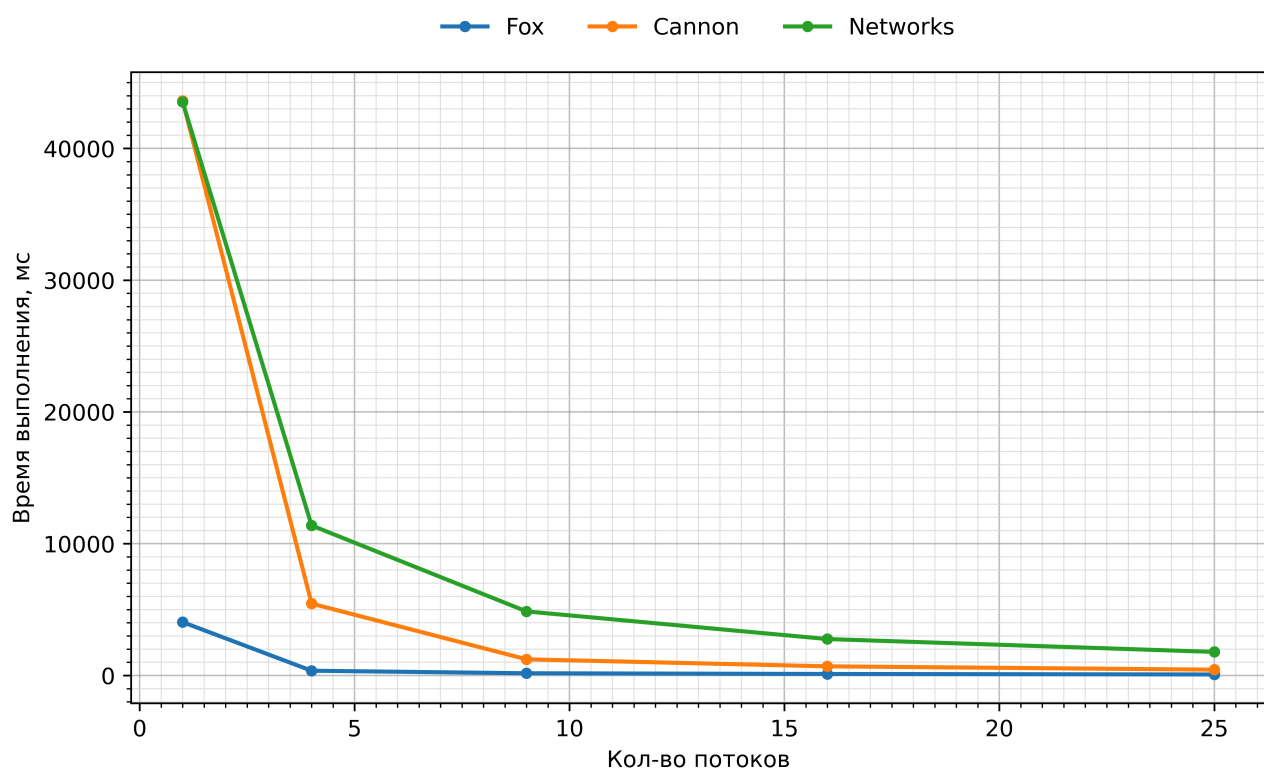


Рисунок 12 – График зависимости времени выполнения перемножения от кол-ва потоков для разных алгоритмов, матрица 2040x2040

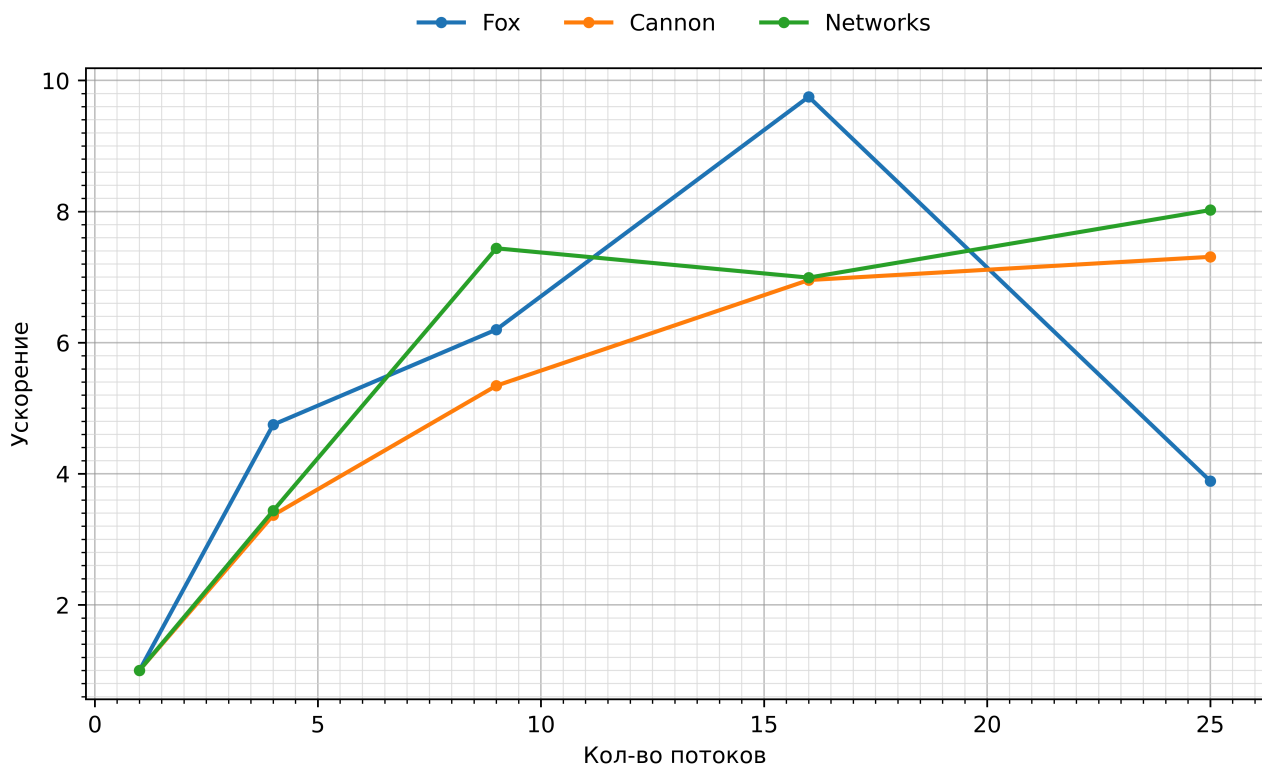


Рисунок 13 – График зависимости ускорения от кол-ва потоков для разных алгоритмов, матрица 480x480

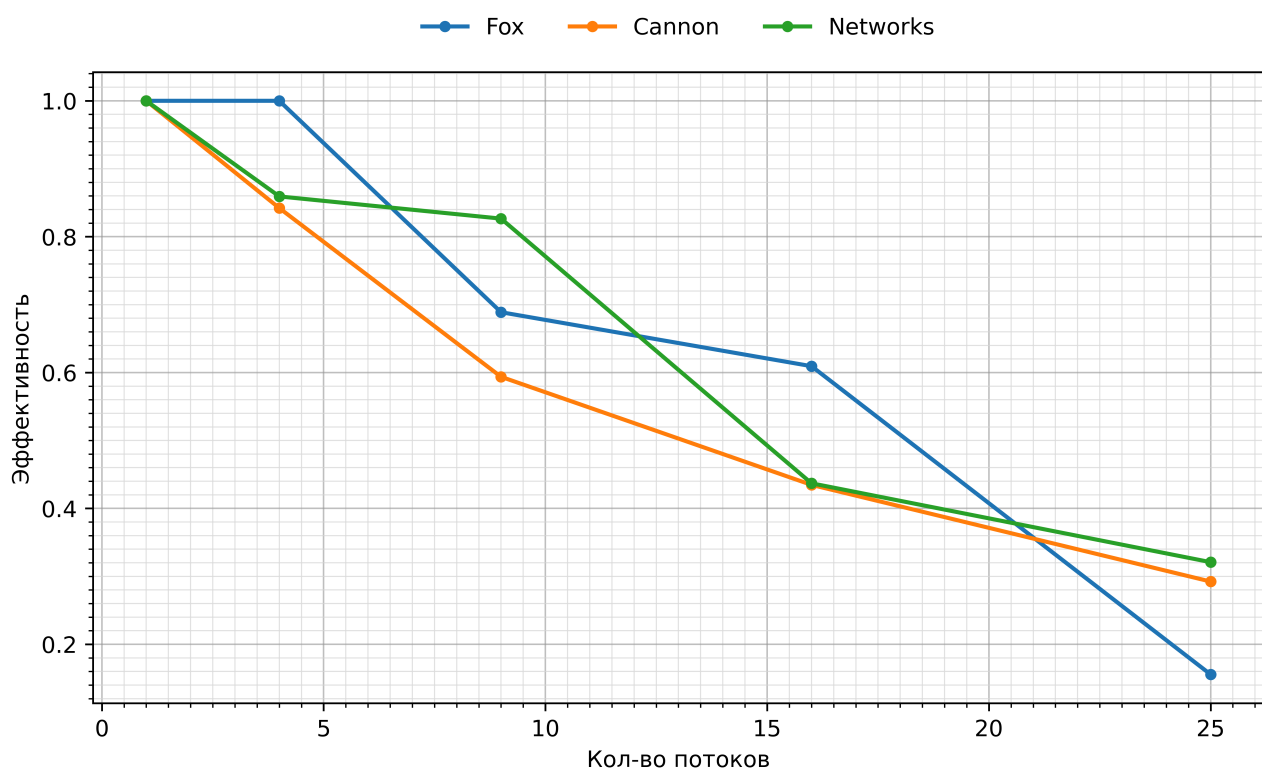


Рисунок 14 – График зависимости эффективности от кол-ва потоков для разных алгоритмов, матрица 480x480

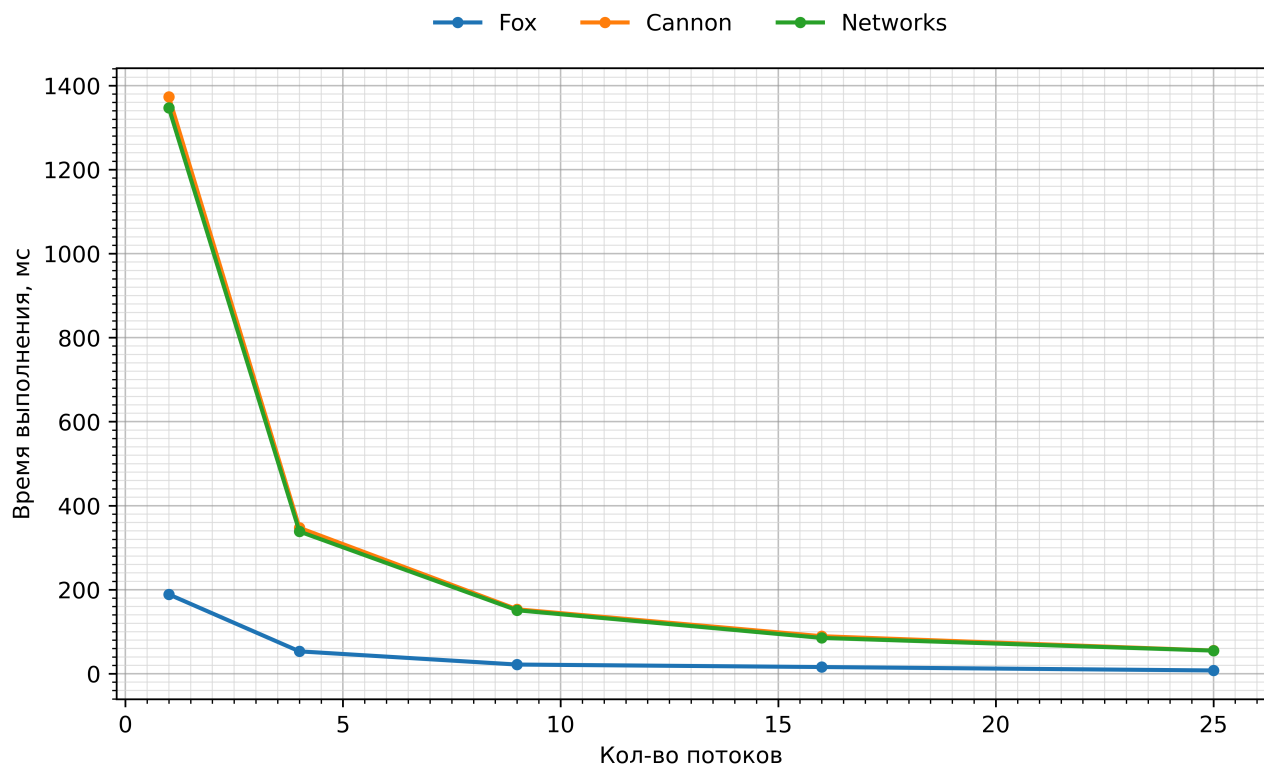


Рисунок 15 – График зависимости времени выполнения перемножения от кол-ва потоков для разных алгоритмов, вторая матрица единичная, размеры матриц 1020x1020

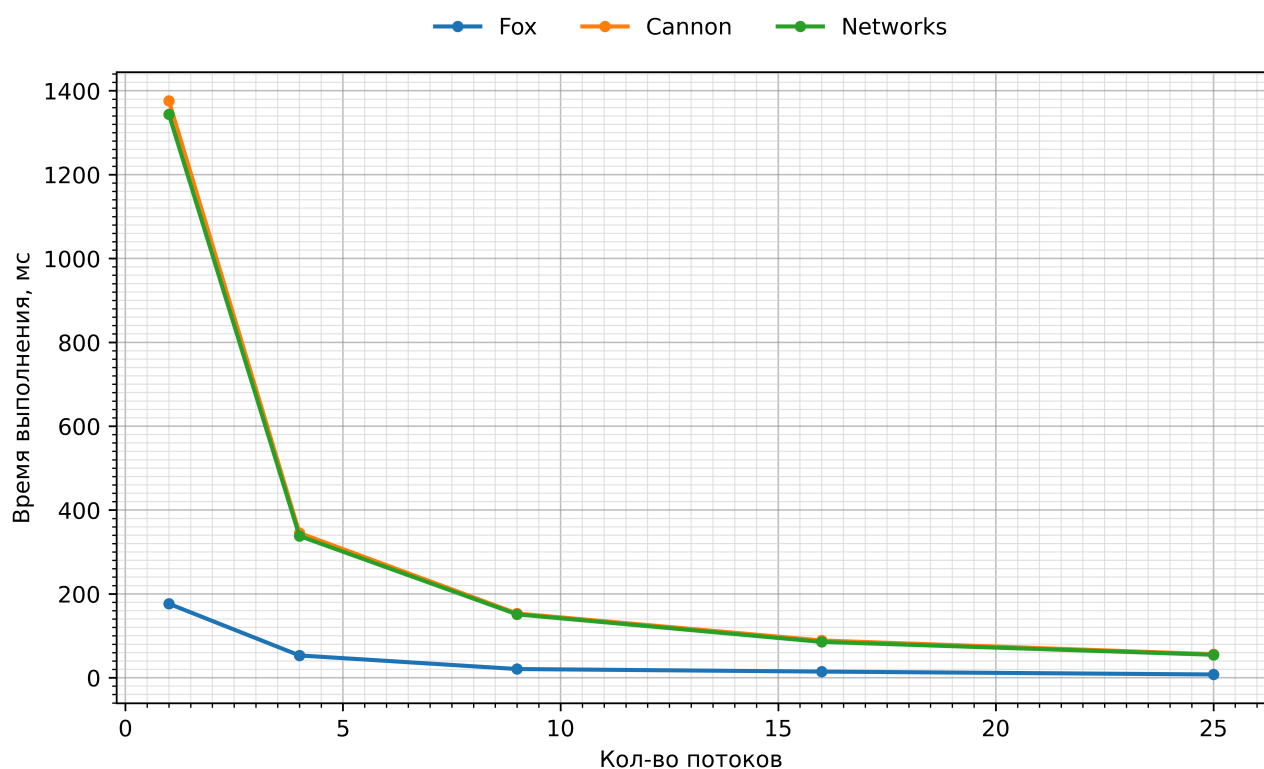


Рисунок 16 – График зависимости времени выполнения перемножения от кол-ва потоков для разных алгоритмов, размеры матриц 1020x1020. Матрицы содержали 75% ненулевых элементов

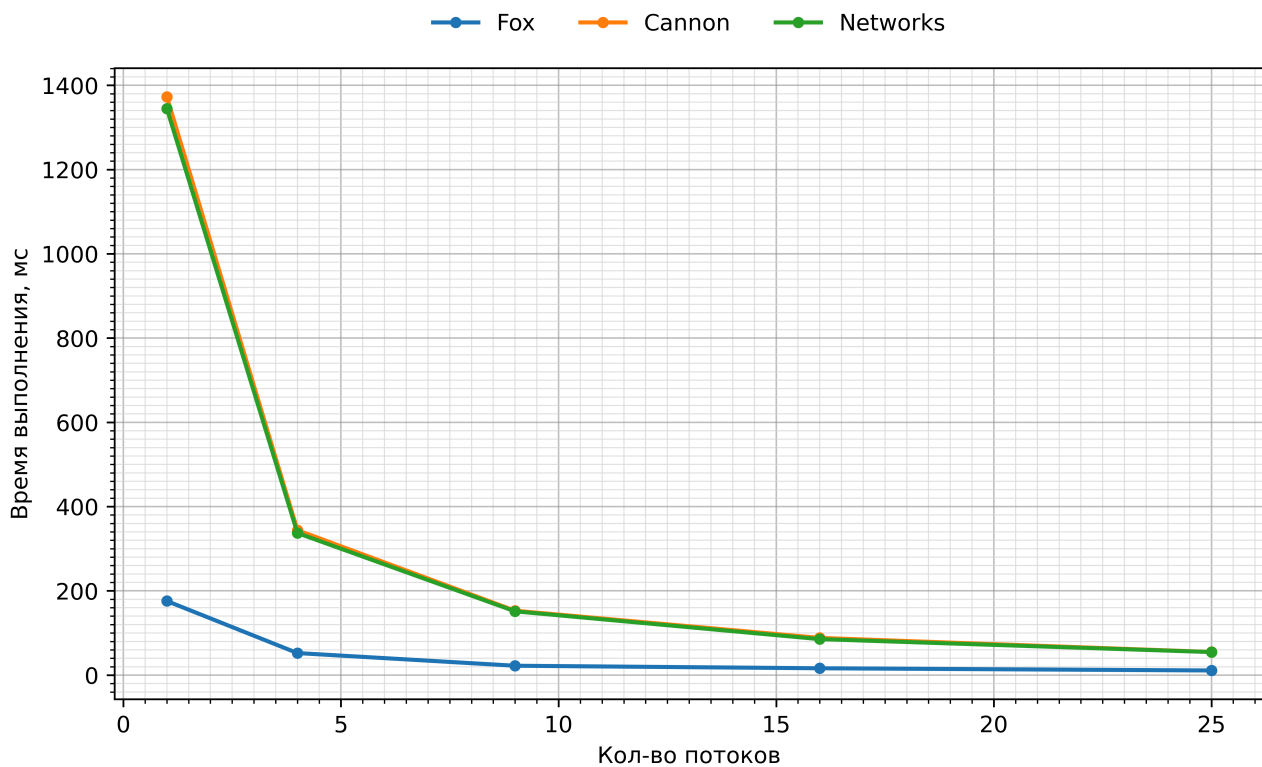


Рисунок 17 – График зависимости времени выполнения перемножения от кол-ва потоков для разных алгоритмов, размеры матриц 1020x1020. Матрицы содержали 25% ненулевых элементов

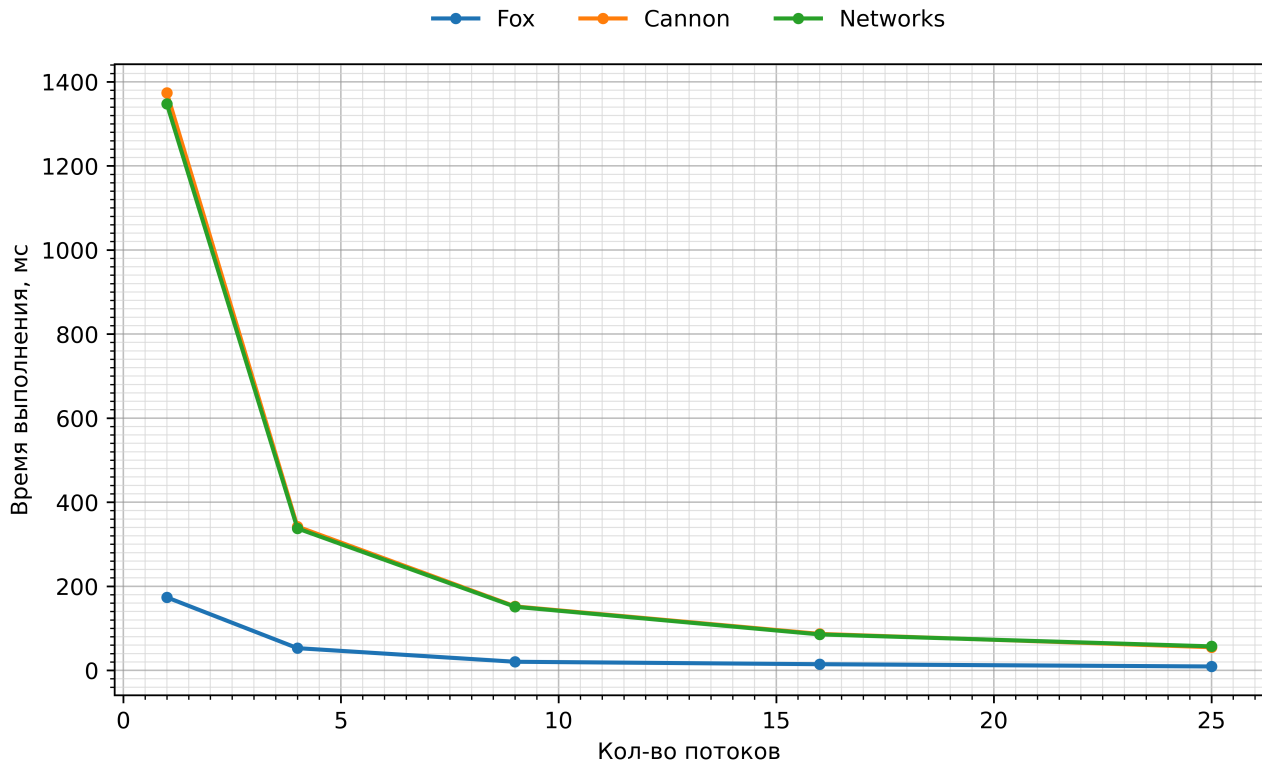


Рисунок 18 – График зависимости времени выполнения перемножения от кол-ва потоков для разных алгоритмов, размеры матриц 1020x1020. Был увеличен разброс случайных чисел (-10млн, 10млн)

6. Заключение

В рамках работы был проведён анализ последовательного и параллельного алгоритмов умножения матриц, а также построена блок-схема алгоритма. На основе полученных экспериментальных данных можно сделать следующие выводы:

Последовательный алгоритм. По графику зависимости времени выполнения от размера матрицы видно, что рост времени выполнения зависит от размеров перемножаемых матриц. Для малых размеров матриц время обработки незначительно, однако при $n = 1024$ наблюдается резкий рост времени. При достижении этого размера матрицы, занимаемая память превысила размер L3 кэша, поэтому появились кэш-промахи, так как матрица перестала целиком помещаться в кэш. Это значительно замедлило выполнение.

Это подтверждает плохую масштабируемость последовательной реализации.

Параллельный алгоритм. Параллельный алгоритм показывает значительный прирост производительности по сравнению с последовательным. При увеличении числа потоков время выполнения снижается, что подтверждает эффективность распараллеливания. Начиная с определённого числа потоков ускорение замедляется из-за накладных расходов и аппаратных ограничений. Эффективность снижается с ростом числа потоков ввиду синхронизационных затрат и неравномерной загрузки потоков.

Максимальное ускорение от параллелизма наблюдается при 25-30 потоках. Эффективность распараллеливания варьируется в пределах 0.7-0.99, в зависимости от количества потоков.

На графике ускорения для параллельного алгоритма наблюдаются аномалии: при увеличении числа потоков до 12–16 ускорение и эффективность резко снижаются, затем остаются примерно постоянными, а после 25 потоков — вновь возрастают.

Анализ аппаратной конфигурации показал, что вычисления выполнялись на виртуальной машине с одним общим L3-кэшем объёмом 16 МБ для всех ядер. Из-за этого все потоки используют общий кэш и общую память, конкурируя за одни и те же ресурсы. При увеличении числа потоков до 12–16 кэш и шины памяти перегружаются — возрастает число промахов кэша, и эффективность

падает. Когда число потоков превышает 24, планировщик перераспределяет их по разным физическим ядрам, уменьшая конкуренцию за кэш и память, поэтому производительность снова начинает расти.

Таким образом, выявленные аномалии объясняются особенностями виртуализированного окружения и ограничениями по кэш-ресурсам, а не ошибками в реализации параллельного алгоритма или средствами OpenMP.

Прямоугольные матрицы. Для прямоугольных матриц эффективность зависит от размеров результирующей матрицы. Если размер результирующей матрицы равен размеру большей стороны исходных матриц, то время выполнения будет в разы больше, чем если бы размер был равен меньшей стороне. Это объясняется спецификой исследуемого алгоритма: для обеих матриц нужно сделать равное кол-во операций, но для малой матрицы нужно сделать меньше итераций внешних циклов, но больше для внутренних, что лучше распараллеливается. Размеры матриц, как и в случае квадратных матриц, влияют на время выполнения, но не влияют на разницу между поворотами прямоугольных матриц.

Наполненность матрицы. Исходя из полученных данных, наполненность матрицы (её разреженность, вид (обычная или единичная) или разброс генерируемых случайных элементов) никак не влияют на время выполнения, т.к. не изменяется кол-во выполнимых операций, а разница во времени, потраченным на арифметические операции, минимальна.

Сравнение с другими алгоритмами. Кроме алгоритма умножения матриц в параллельных сетях, от других команд были получены реализации алгоритмов Фокса и Кэннона. Сравнение производилось только на кол-вах потоков, которые подходят для полученных алгоритмов.

На малых матрицах разница между алгоритмами мала, так как затраты на синхронизацию больше, чем время выполнение матрицы. Для остальных размеров квадратных матриц самым эффективным алгоритмом на 1-9 потоках оказался алгоритм Фокса. Алгоритм Кэннона и перемножение в сетях показали практически идентичные результаты, кроме самого большого размера матрицы (2040x2040), на котором алгоритм Кэннона оказался быстрее. На 16-25 потоках все алгоритмы показали схожие результаты. Неэффективность алгоритма

перемножения в сетях невелируется его повышенным ускорением на большом количестве потоков.

Таким образом, самым быстрым на всех размерах оказался алгоритм Фокса. На больших размерах алгоритм Кэннона показывает схожие результаты. Алгоритм перемножения матриц параллельных в сетях имеет наивысшее ускорение при распараллеливании.

7. Приложение

7.1. Программные коды

```
#include <vector>
#include <memory>
#include <random>
#include <algorithm>
#include "mtx.h"

Matrix GenerateRandom(size_t rows, size_t cols, double from = -10, double to = 10) {
    static std::mt19937 gen(std::random_device{}());
    std::uniform_real_distribution<double> value(from, to);

    Matrix m(rows, cols);
    for (size_t i = 0; i < rows * cols; ++i)
        m.ptr[i] = value(gen);
    return m;
}

Matrix GenerateIdentity(size_t size) {
    Matrix m(size, size);
    m.fill(0.0);
    for (size_t i = 0; i < size; ++i)
        m.at(i, i) = 1.0;
    return m;
}

Matrix GenerateSparse(size_t rows, size_t cols, double density = 0.05,
                     double from = -10, double to = 10) {
    static std::mt19937 gen(std::random_device{}());
    std::uniform_real_distribution<double> prob(0.0, 1.0);
    std::uniform_real_distribution<double> value(from, to);

    Matrix m(rows, cols);
    for (size_t i = 0; i < rows * cols; ++i)
        m.ptr[i] = (prob(gen) < density) ? value(gen) : 0.0;
    return m;
}
```

Файл 1 – Генератор матриц

```
#include "mtx.h"
int mul_matrix(const Matrix* matr_1, const Matrix* matr_2, Matrix* res_m, int threads){
    if(matr_1->Columns != matr_2->Rows) return 1;
    if(res_m->Rows != matr_1->Rows) return 1;
    if(res_m->Columns != matr_2->Columns) return 1;

    #pragma omp parallel for shared(res_m, matr_1, matr_2) default(none) num_threads(threads)
    for(size_t i = 0; i < res_m->Rows; i++){
        for(size_t j = 0; j < res_m->Columns; j++){
            double sum = 0;
            for(size_t p = 0; p < matr_1->Columns; p++){
                sum += matr_1->ptr[i * matr_1->Columns + p] * matr_2->ptr[p * matr_2->Columns + j];
            }
        }
    }
```

```

        res_m->ptr[i * res_m->Columns + j] = sum;
    }
}
return 0;
}

int mul_matrix_seq(const Matrix* matr_1, const Matrix* matr_2, Matrix* res_m){
    if(matr_1->Columns != matr_2->Rows) return 1;
    if(res_m->Rows != matr_1->Rows) return 1;
    if(res_m->Columns != matr_2->Columns) return 1;

    for(size_t i = 0; i < res_m->Rows; i++){
        for(size_t j = 0; j < res_m->Columns; j++){
            double sum = 0;
            for(size_t p = 0; p < matr_1->Columns; p++){
                sum += matr_1->ptr[i * matr_1->Columns + p] * matr_2->ptr[p * matr_2->Columns + j];
            }
            res_m->ptr[i * res_m->Columns + j] = sum;
        }
    }
    return 0;
}

```

Файл 2 – Реализации алгоритмов умножения матриц

```

#include <stdlib>
#include <cmath>
#include <omp.h>
#include "mtx.h"

// === КЛАССИЧЕСКИЙ АЛГОРИТМ ФОКСА ТОЛЬКО ДЛЯ ПОЛНЫХ КВАДРАТОВ === by mr. Semen
void FoxParallelMultiplication(double* A, double* B, double* C, int Size, int NumThreads) {
    int q = (int)sqrt((double)NumThreads);

    // Проверяем, является ли число потоков полным квадратом и делит ли размер на q
    if (q * q == NumThreads && Size % q == 0 && q > 0) {
        int BlockSize = Size / q;

#pragma omp parallel num_threads(NumThreads)
        {
            int tid = omp_get_thread_num();
            int row = tid / q;
            int col = tid % q;

            double* Cblock = (double*)calloc((size_t)BlockSize * BlockSize, sizeof(double));
            double* TempA = (double*)malloc((size_t)BlockSize * BlockSize * sizeof(double));
            double* TempB = (double*)malloc((size_t)BlockSize * BlockSize * sizeof(double));

            for (int iter = 0; iter < q; iter++) {
                int pivot = (row + iter) % q;

                // Копирование блока A
                for (int bi = 0; bi < BlockSize; bi++) {
                    for (int bj = 0; bj < BlockSize; bj++) {
                        int gi = row * BlockSize + bi;
                        int gj = pivot * BlockSize + bj;

```

```

        TempA[bi * BlockSize + bj] = A[gi * Size + gj];
    }
}

// Копирование блока B
for (int bi = 0; bi < BlockSize; bi++) {
    for (int bj = 0; bj < BlockSize; bj++) {
        int gi = pivot * BlockSize + bi;
        int gj = col * BlockSize + bj;
        TempB[bi * BlockSize + bj] = B[gi * Size + gj];
    }
}

// Умножение блоков
for (int i = 0; i < BlockSize; i++) {
    for (int k = 0; k < BlockSize; k++) {
        double a_val = TempA[i * BlockSize + k];
        for (int j = 0; j < BlockSize; j++) {
            Cblock[i * BlockSize + j] += a_val * TempB[k * BlockSize + j];
        }
    }
}

// Запись результата
for (int bi = 0; bi < BlockSize; bi++) {
    for (int bj = 0; bj < BlockSize; bj++) {
        int gi = row * BlockSize + bi;
        int gj = col * BlockSize + bj;
        C[gi * Size + gj] = Cblock[bi * BlockSize + bj];
    }
}

free(Cblock);
free(TempA);
free(TempB);
}
}

// Алгоритм Кэннона (параллельная версия) by ms. Nadya
void cannon_par(double *A, double *B, double *C, int n, int blocks_per_line) {
    int threads = blocks_per_line * blocks_per_line;

    #pragma omp parallel for collapse(2) num_threads(threads) default(none) shared(A, B, C,
n, blocks_per_line)
    for (int block_i = 0; block_i < blocks_per_line; block_i++) {
        for (int block_j = 0; block_j < blocks_per_line; block_j++) {

            int block_size = n / blocks_per_line;
            double* local_C = (double*)calloc(block_size * block_size, sizeof(double));

            for (int step = 0; step < blocks_per_line; step++) {

                // Сдвиги: начальный + текущий шаг
                int shifted_A_j = (block_j + block_i + step) % blocks_per_line;
                int shifted_B_i = (block_i + block_j + step) % blocks_per_line;

```

```

int start_i_A = block_i * block_size;
int start_j_A = shifted_A_j * block_size;

int start_i_B = shifted_B_i * block_size;
int start_j_B = block_j * block_size;

int start_i_C = start_i_A;
int start_j_C = start_j_B;

// Умножение блока AxB и накопление в C
for (int i = 0; i < block_size; i++) {
    for (int j = 0; j < block_size; j++) {
        double sum = 0.0;
        for (int k = 0; k < block_size; k++) {
            int idx_A = (start_i_A + i) * n + (start_j_A + k);
            int idx_B = (start_i_B + k) * n + (start_j_B + j);
            sum += A[idx_A] * B[idx_B];
        }

        int idx_C = (start_i_C + i) * n + (start_j_C + j);
        // Обновляем C с учетом суммы по этому k-блоку
#pragma omp atomic
        C[idx_C] += sum;
    }
}
}
}
}
}
}

```

Файл 3 – Реализации алгоритмов умножения матриц от других команд

```

#include <vector>
#include <fstream>
#include <memory>
#include <string>
#include <filesystem>
#include <utility>
#include <iostream>

std::vector<std::pair<int, int>> threadsAndBlockSize = {
    {1, 1},
    {4, 2},
    {9, 3},
    {16, 4},
    {25, 5}
};

int threads = 32;
int repeats = 20;

struct TGraphDataSq {
    size_t rows;
    size_t cols;
    std::string filename;
};

```

```

std::unique_ptr<std::ofstream> file;

TGraphDataSq(size_t rows_, size_t cols_, const std::string& filename)
    : rows(rows_), cols(cols_), filename(filename)
{
    std::filesystem::create_directories(std::filesystem::path(filename).parent_path());
    file = std::make_unique<std::ofstream>(filename);
    if (!file->is_open()) {
        std::cerr << "Ошибка: не удалось открыть файл " << filename << "\n";
        return;
    }
    *file << "3\n"
        << "Fox\n"
        << "Cannon\n"
        << "Networks\n"
        << "Кол-во потоков\n"
        << "Время выполнения, мс\n"
        << "none\n";
}

TGraphDataSq(const TGraphDataSq&) = delete;
TGraphDataSq& operator=(const TGraphDataSq&) = delete;
TGraphDataSq(TGraphDataSq&&) noexcept = default;
TGraphDataSq& operator=(TGraphDataSq&&) noexcept = default;

template <typename T>
TGraphDataSq& operator<<(const T& value) {
    if (file && file->is_open()) {
        *file << value;
    }
    return *this;
}

};

struct TGraphDataRec {
    std::vector<std::pair<size_t, size_t>> mtxs;
    std::string filename;
    std::unique_ptr<std::ofstream> file;

    TGraphDataRec(const std::vector<std::pair<size_t, size_t>>& mtxs_, const std::string& filename_)
        : mtxs(mtxs_), filename(filename_)
    {
        std::filesystem::create_directories(std::filesystem::path(filename_).parent_path());
        file = std::make_unique<std::ofstream>(filename_);
        if (!file->is_open()) {
            std::cerr << "Ошибка: не удалось открыть файл " << filename_ << "\n";
            return;
        }
        *file << mtxs.size() << "\n";
        for (const auto& mtx : mtxs) {
            *file << mtx.first << "x" << mtx.second << "\n";
        }
        *file << "Кол-во потоков\n"
            << "Время выполнения, мс\n"
            << "none\n";
    }
}

```

```

TGraphDataRec(const TGraphDataRec&) = delete;
TGraphDataRec& operator=(const TGraphDataRec&) = delete;
TGraphDataRec(TGraphDataRec&&) noexcept = default;
TGraphDataRec& operator=(TGraphDataRec&&) noexcept = default;

template <typename T>
TGraphDataRec& operator<<(const T& value) {
    if (file && file->is_open()) {
        *file << value;
    }
    return *this;
}

};

std::vector<TGraphDataSq> allSizesSq;
std::vector<TGraphDataRec> allRecVec;
void init_vectors() {
    allSizesSq.emplace_back(60, 60, "results/sq1_small");
    allSizesSq.emplace_back(480, 480, "results/sq2_middle");
    allSizesSq.emplace_back(1020, 1020, "results/sq3_large");
    allSizesSq.emplace_back(2040, 2040, "results/sq4_mega");

    allRecVec.emplace_back(
        std::vector<std::pair<size_t, size_t>>{ {1024, 64}, {64, 1024}, {256, 256} },
        "results/rec1_small"
    );
    allRecVec.emplace_back(
        std::vector<std::pair<size_t, size_t>>{ {2048, 124}, {124, 2048}, {512, 512} },
        "results/rec2_big"
    );
}

TGraphDataSq identitySq = {1020, 1020, "results/sq5_identity"};
TGraphDataSq sparseSq = {1020, 1020, "results/sq6_sparse"};
TGraphDataSq sparseLowSq = {1020, 1020, "results/sq7_sparse_low"};
TGraphDataSq randomSq = {1020, 1020, "results/sq8_bigrandom"};

TGraphDataRec sparseRec = {
    {
        {64, 1024},
        {1024, 64},
        {512, 512}
    },
    "results/rec3_sparse"
};
};

```

Файл 4 – Начальные данные для экспериментов

```

#include "external.h"
#include "mul_matrix.h"
#include "gen.h"
#include "data.h"

double MeasureFuncSquare(int func_num, Matrix& a, Matrix& b, Matrix& c, int t, int blocks) {
    double sumtime = 0;
    double start, end;

```

```

for (int i=0; i < repeats; ++i) {
    switch (func_num)
    {
        case 0:
            start = omp_get_wtime();
            FoxParallelMultiplication(a.ptr.get(), b.ptr.get(), c.ptr.get(), a.Rows, t);
            end = omp_get_wtime();
            break;
        case 1:
            start = omp_get_wtime();
            cannon_par(a.ptr.get(), b.ptr.get(), c.ptr.get(), a.Rows, blocks);
            end = omp_get_wtime();
            break;
        case 2:
            start = omp_get_wtime();
            mul_matrix(&a, &b, &c, t);
            end = omp_get_wtime();
            break;
    }
    sumtime += end - start;
}
sumtime /= repeats;
sumtime *= 1000;
return sumtime;
}

void TestForSquare(TGraphDataSq& squareData, Matrix& a, Matrix& b, Matrix& c) {
    for (const auto& [t, block] : threadsAndBlockSize) {
        std::cout << "finished " << t << " threads" << std::endl;
        squareData << t << " ";
        for (int func_num = 0; func_num < 3; ++func_num) {
            c.fill(0);
            squareData << MeasureFuncSquare(func_num, a, b, c, t, block) << " ";
        }
        squareData << "\n";
    }
}

struct Dataset {
    Matrix a;
    Matrix b;
    Matrix c;
};

void TestForRect(TGraphDataRec& recData, std::vector<Dataset>& data) {
    double start, end;
    for (int t = 1; t <= threads; ++t) {
        recData << t << " ";
        for (auto& dataset : data) {
            double sumtime = 0;
            for (int i=0; i < repeats; ++i) {
                dataset.c.fill(0);
                start = omp_get_wtime();
                mul_matrix(&dataset.a, &dataset.b, &dataset.c, t);
                end = omp_get_wtime();
                sumtime += end - start;
            }
        }
    }
}

```

```

        sumtime /= repeats;
        sumtime *= 1000;
        recData << sumtime << " ";
    }
    recData << "\n";
}

void sequential_exp() {
    std::vector<int> seq_sizes = {
        32, 142, 252, 363, 473,
        584, 694, 804, 914, 1024
    };

    std::ofstream file("results/seq_res");

    file << "1\n" << "Последовательная реализация\n" << "Кол-во строк в кв. matr.\n" << "Время
выполнения, мс\n" << "none\n";

    double T1;
    double start, end;
    for (int size : seq_sizes) {
        Matrix a = GenerateRandom(size, size);
        Matrix b = GenerateRandom(size, size);
        Matrix c(size, size);

        double sum_time = 0;
        for (int i=0; i < repeats; ++i) {
            c.fill(0.0);
            start = omp_get_wtime();
            mul_matrix_seq(&a, &b, &c);
            end = omp_get_wtime();
            sum_time += end - start;
        }
        sum_time /= repeats;
        sum_time *= 1000;
        file << size << " " << sum_time << "\n";
    }
}

void multithread_base_exp() {
    size_t size = 1024;
    std::ofstream file("results/mtt_res");
    std::ofstream file_speed("results/mtt_speed");
    std::ofstream file_eff("results/mtt_eff");

    file << "1\n" << "Параллельная реализация\n" << "Кол-во потоков\n" << "Время выполнения, мс\n"
<< "none\n";
    file_speed << "1\n" << "Параллельная реализация\n" << "Кол-во потоков\n" << "Ускорение\n"
<< "none\n";
    file_eff << "1\n" << "Параллельная реализация\n" << "Кол-во потоков\n" << "Эффективность\n"
<< "none\n";

    Matrix a = GenerateRandom(size, size);
    Matrix b = GenerateRandom(size, size);
    Matrix c(size, size);

```

```

double T1;
double start, end;
for (int t=1; t <= threads; ++t) {
    double sum_time = 0;
    for (int i=0; i < repeats; ++i) {
        c.fill(0.0);
        start = omp_get_wtime();
        mul_matrix(&a, &b, &c, t);
        end = omp_get_wtime();
        sum_time += end - start;
    }
    sum_time /= repeats;
    sum_time *= 1000;
    if (t == 1) {
        T1 = sum_time;
    }
    file << t << " " << sum_time << "\n";
    file_speed << t << " " << T1 / sum_time << "\n";
    file_eff << t << " " << T1/sum_time/t << "\n";
}
}

int main() {
    sequential_exp();
    std::cout << "finished seq " << std::endl;
    multithread_base_exp();
    std::cout << "finished mtt" << std::endl;
    init_vectors();
    for (auto& squareData: allSizesSq) {
        Matrix a = GenerateRandom(squareData.rows, squareData.cols);
        Matrix b = GenerateRandom(squareData.rows, squareData.cols);
        Matrix c(squareData.rows, squareData.cols);
        TestForSquare(squareData, a, b, c);
        std::cout << "finished " << squareData.rows << std::endl;
    }
    std::cout << "finished square" << std::endl;
    { // identity
        Matrix a = GenerateRandom(identitySq.rows, identitySq.cols);
        Matrix b = GenerateIdentity(identitySq.rows);
        Matrix c(identitySq.rows, identitySq.cols);
        TestForSquare(identitySq, a, b, c);
    }
    std::cout << "finished identity" << std::endl;
    { // sparse (75% values)
        Matrix a = GenerateRandom(sparseSq.rows, sparseSq.cols, 0.75);
        Matrix b = GenerateSparse(sparseSq.rows, sparseSq.cols, 0.75);
        Matrix c(sparseSq.rows, sparseSq.cols);
        TestForSquare(sparseSq, a, b, c);
    }
    std::cout << "finished sparse" << std::endl;
    { // sparse (25% values)
        Matrix a = GenerateRandom(sparseLowSq.rows, sparseLowSq.cols, 0.25);
        Matrix b = GenerateSparse(sparseLowSq.rows, sparseLowSq.cols, 0.25);
        Matrix c(sparseLowSq.rows, sparseLowSq.cols);
        TestForSquare(sparseLowSq, a, b, c);
    }
    std::cout << "finished sparse low" << std::endl;
}

```

```

{ // bigRand1
    Matrix a = GenerateRandom(randomSq.rows, randomSq.cols, -10000000.0, 10000000.0);
    Matrix b = GenerateRandom(randomSq.rows, randomSq.cols, -10000000.0, 10000000.0);
    Matrix c(randomSq.rows, randomSq.cols);
    TestForSquare(randomSq, a, b, c);
}
std::cout << "finished bigrand" << std::endl;
// rectangles
for (auto& recData: allRecVec) {
    std::vector<Dataset> data;
    for (const auto& [rows, cols] : recData.mtxs) {
        data.emplace_back(
            GenerateRandom(rows, cols),
            GenerateRandom(cols, rows),
            Matrix(rows, rows)
        );
    }
    TestForRect(recData, data);
    std::cout << "finished " << recData.mtxs[0].first << std::endl;
}
std::cout << "finished rect" << std::endl;
// sparsed rectangles
std::vector<Dataset> data;
for (const auto& [rows, cols] : sparseRec.mtxs) {
    data.emplace_back(
        GenerateRandom(rows, cols),
        GenerateRandom(cols, rows),
        Matrix(rows, rows)
    );
}
TestForRect(sparseRec, data);
}

```

Файл 5 – Измерение времени работы алгоритмов

7.2. Таблицы

Таблица 1 – Результаты измерений для последовательного алгоритма

Кол-во строк в кв. матр.	Последовательная реализация
32	0.0172377
142	2.22286
252	16.3339
363	54.6704
473	123.306
584	238.161
694	407.589
804	644.217
914	958.205
1024	6358.09

Таблица 2 – Результаты измерений для параллельного алгоритма на квадратной матрице размером 1024

Кол-во потоков	Параллельная реализация
1	6349.23
2	3197.25
3	2129.62
4	1602.84
5	1285.04
6	1083.29
7	922.398
8	815.619
9	724.087
10	654.736
11	594.587
12	581.435
13	578.605
14	646.295
15	801.094
16	759.56
17	746.38
18	766.538
19	773.416
20	763.033
21	770.358
22	757.744
23	714.599
24	700.264
25	492.487
26	285.039
27	330.028
28	259.989
29	270.844
30	265.673
31	262.957
32	417.185

Таблица 3 – Результаты измерений для параллельного алгоритма на прямоугольных матрицах

Кол-во потоков	2048x124	124x2048	512x512
1	1749.95	40.717	460.818
2	876.112	20.4478	231.394
3	598.816	13.897	160.553
4	451.087	10.4262	121.131
5	362.958	8.49843	96.5015
6	301.495	7.19327	79.1861
7	257.916	6.2767	71.8707
8	239.563	5.44224	62.551
9	211.004	4.85462	53.668
10	177.842	4.4655	47.0682
11	165.727	4.30182	44.3208
12	153.199	3.82313	41.5025
13	140.781	3.56891	37.4926
14	130.559	3.36828	35.9705
15	124.648	3.34529	34.4899
16	115.72	2.8955	32.3562
17	109.151	3.02213	30.1392
18	103.702	2.61502	28.2644
19	97.8586	2.63892	27.8044
20	94.2527	2.66807	25.8873
21	89.0492	2.27696	25.1437
22	84.0643	2.30206	22.8122
23	80.3797	2.18396	22.1935
24	78.438	2.22754	21.342
25	74.6887	2.0327	19.9588
26	71.0583	1.84618	19.0377
27	71.8249	2.18337	18.9782
28	70.9181	2.29516	19.7667
29	72.5563	2.37963	20.0612
30	74.3327	2.66639	20.4431
31	73.5023	1.47864	20.3024
32	72.9309	2.72344	20.7223
5111			

Таблица 4 – Результаты измерений для разных параллельных алгоритмов на квадратной матрице размером 2040

Кол-во потоков	Fox	Cannon	Networks
1	4046.12	43612.3	43528.2
4	359.335	5451.65	11383.4
9	166.859	1226.17	4859.68
16	113.053	692.332	2763.53
25	74.7778	439.39	1786.99

Таблица 5 – Результаты измерений для разных параллельных алгоритмов для второй единичной матрицы размером 1020

Кол-во потоков	Fox	Cannon	Networks
1	188.818	1373.3	1346.77
4	53.2438	347.346	338.469
9	21.84	153.366	151.097
16	15.921	89.3249	85.24
25	7.73894	55.2201	55.0184

Таблица 6 – Результаты измерений для разных параллельных алгоритмов на разреженной матрице размером 1020 (25% ненулевых)

Кол-во потоков	Fox	Cannon	Networks
1	175.858	1372.64	1344.62
4	52.3401	344.055	337.169
9	22.2717	152.885	151.413
16	16.2575	88.3561	85.431
25	11.0324	54.971	54.6999

Таблица 7 – Результаты измерений для разных параллельных алгоритмов на матрице размером 1020 с увеличенным разбросом случайных чисел

Кол-во потоков	Fox	Cannon	Networks
1	173.665	1373.82	1347.3
4	52.8648	342.008	337.638
9	20.4757	152.197	151.298
16	14.6563	86.7358	85.4986
25	9.26895	55.084	57.0593